

DiverFPS: Generating Diverse Solutions for Floating-Point SMT Formulas

SHUANGYU LYU, Beihang University, China

CHUAN LUO*, Beihang University, China

RUIZHI SHI, Beihang University, China

ZHUO SU, Beihang University, China

CHUNMING HU, Beihang University, China

Satisfiability Modulo Theories (SMT) is a fundamental technique underpinning a wide range of applications in software engineering and testing. Among various SMT theories, the theory of floating-point plays a crucial role in practical software systems, yet reasoning about floating-point constraints remains challenging. Although existing SMT solvers are capable of producing single solutions, many applications, particularly in software testing and verification, require diverse sets of solutions to adequately exercise program behaviors. While achieving high diversity is essential for exploring system behaviors, it is also important to limit the number of generated solutions, as overly large solution sets can considerably increase testing time and resource consumption, thereby diminishing efficiency. In this work, we propose *DiverFPS*, a novel floating-point SMT sampler designed to generate small solution sets that achieve high target abstract syntax tree (AST)-coverage, which is commonly regarded as the standard metric for assessing solution diversity in the SMT sampling domain. *DiverFPS* operates in two stages: an exploration stage that aims to achieve the target AST-coverage as fully as possible, and a pruning stage that prunes redundant solutions while preserving the target AST-coverage. We further introduce three novel techniques, namely solution-driven restart strategy, context-aware encoding technology, and coverage-driven pruning strategy, which enhance the performance of *DiverFPS*. Extensive experiments on publicly available SMT-LIB benchmarks for QF_FP and QF_BVFP logics demonstrate that *DiverFPS* outperforms state-of-the-art SMT samplers. It successfully achieves the target AST-coverage on a larger number of benchmarks, which existing samplers fail to reach, while producing solution sets that are up to 92.9% smaller, the former improving the effectiveness of practical software testing and verification, and the latter improving testing efficiency and reducing resource consumption. These results demonstrate that *DiverFPS* is a high-performing sampler for floating-point SMT sampling.

CCS Concepts: • **Software and its engineering** → **Formal software verification**; • **Theory of computation** → **Logic and verification**.

Additional Key Words and Phrases: Satisfiability Modulo Theories, Sampling

ACM Reference Format:

Shuangyu Lyu, Chuan Luo, Ruizhi Shi, Zhuo Su, and Chunming Hu. 2026. DiverFPS: Generating Diverse Solutions for Floating-Point SMT Formulas. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE053 (July 2026), 24 pages. <https://doi.org/10.1145/3797081>

*Corresponding author.

Authors' Contact Information: [Shuangyu Lyu](mailto:shuangyulyu@buaa.edu.cn), Beihang University, Beijing, China, shuangyulyu@buaa.edu.cn; [Chuan Luo](mailto:chuanluo@buaa.edu.cn), Beihang University, Beijing, China, chuanluo@buaa.edu.cn; [Ruizhi Shi](mailto:shirui@buaa.edu.cn), Beihang University, Beijing, China, shirui@buaa.edu.cn; [Zhuo Su](mailto:suzcpp@buaa.edu.cn), Beihang University, Beijing, China, suzcpp@buaa.edu.cn; [Chunming Hu](mailto:hucm@buaa.edu.cn), Beihang University, Beijing, China, hucm@buaa.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE053

<https://doi.org/10.1145/3797081>

1 Introduction

Satisfiability Modulo Theories (SMT) has become a fundamental technique that underpins a wide range of applications in software engineering and software testing [6, 14, 22, 23, 25, 27, 54]. Among the various SMT theories, the theory of floating-point is of great importance due to its widespread use in diverse software systems [13, 21, 34]. Nevertheless, reasoning about floating-point constraints remains particularly challenging [7, 21, 39], and has been the subject of a lot of recent and ongoing research [7, 13, 21, 34, 39, 49]. Errors in this domain can have catastrophic consequences [1, 55], resulting in severe human casualties or substantial economic losses [1, 55]. Accordingly, this work focuses on the theory of floating-point.

Although state-of-the-art SMT solvers can determine the satisfiability of SMT formulas and produce a single solution [7, 13, 16, 21, 34, 39, 49], many practical applications, particularly in software testing and verification, demand diverse solutions rather than just one [7, 17, 53]. For example, symbolic execution can benefit from extracting multiple solutions from constraints instead of enumerating multiple program paths from a certain program point [7, 19, 30, 41]. In software testing, diverse solutions naturally translate into diverse test cases, which in turn help uncover inputs that expose safety violations or erroneous behavior [29, 43–45, 56]. While diversity of solutions is essential for effective software testing and verification, it must be carefully balanced against the efficiency of these processes. Executing a large number of test cases (*i.e.*, solutions) can be prohibitively expensive, both in terms of time and resources [45]. In complex software systems, executing a single test case may already take several days [52], which can substantially slow down the development and testing cycle [51]. Consequently, numerous studies have emphasized the importance of reducing the number of test cases [12, 15, 26, 32, 45, 51]. This leads to the SMT sampling problem, which seeks to construct small yet diverse sets of solutions that exercise a wide range of behaviors in software systems, thereby facilitating the detection of hidden faults [7, 17, 53].

To assess diversity, some studies have used the uniformity of the solution distribution as a proxy [19, 28]. However, obtaining a precise estimate of distribution uniformity requires sampling a large number of solutions, making it unsuitable for large-scale SMT formulas. Moreover, research [17, 43, 44] has indicated that uniform sampling is not effective in software testing. Consequently, recent works on SMT sampling have employed abstract syntax tree (AST)-coverage as the primary metric [7, 17, 53], and in the context of floating-point sampling, AST-coverage has been adopted as the main optimization objective [7]. As formally defined in Section 2, AST-coverage reflects the extent of syntactic structures and decision branches exercised within a formula, which naturally induces diverse behaviors in programs and software systems, making it a meaningful proxy for diversity [17, 53]. Furthermore, prior work [17] reports that solution sets with high AST-coverage lead to improved fault detection capability. Therefore, based on these two important metrics, AST-coverage and solution set size, the goal of SMT sampling for a given formula is to generate a solution set that achieves the target AST-coverage while minimizing its size.

Despite its importance, research on SMT sampling remains limited. Most existing SMT samplers have focused on bit-vector theories and their extensions [17, 18, 59], or on integer arithmetic [53]. In contrast, sampling for floating-point formulas is more challenging: even obtaining a single solution for a floating-point constraint can be inefficient [7, 39] due to the complexity of floating-point reasoning, and generating multiple diverse solutions further increases the difficulty. Moreover, many strategies effective for bit-vector or integer theories do not directly extend to floating-point formulas because of their unique semantics and reasoning characteristics [7, 39].

To efficiently handle floating-point constraints, [39] proposed a coverage-guided fuzzing method for solving floating-point SMT formulas, which has been empirically shown to be effective for floating-point constraints [39]. While this method is incomplete, as it can only find solutions for

satisfiable formulas and cannot prove unsatisfiability [39], and it exhibits limitations when solving complex formulas with very few solutions, these issues are less relevant in the context of SMT sampling, where formulas are typically satisfiable and have many diverse solutions. Building on this coverage-guided fuzzing SMT solver, a recent study [7] introduced *jFSampler*, a sampler specifically designed for floating-point SMT sampling. According to [7], it can rapidly generate a large number of solutions for floating-point SMT formulas. However, it still suffers from two critical challenges: the exploration bottleneck, *i.e.*, the inability to achieve sufficiently high target AST-coverage, and the redundancy issue, *i.e.*, the generation of unnecessarily large solution sets containing duplicate or uninformative solutions. Our experimental results in Section 5 further confirm these limitations. These challenges restrict the practical applicability of the method: the failure to achieve target AST-coverage reduces the fault detection capability, while the generation of redundant solutions can considerably decrease the efficiency of software testing and verification.

In this work, we introduce *DiverFPS*, a novel SMT sampler for floating-point constraints. *DiverFPS* operates in two stages: the exploration stage and the pruning stage. During the exploration stage, *DiverFPS* systematically explores the solution space of an SMT formula in an effort to achieve the target AST-coverage. In the pruning stage, *DiverFPS* removes redundant solutions while preserving the achieved target AST-coverage by prioritizing solutions according to their contribution to AST-coverage. In addition, we propose three novel techniques: solution-driven restart strategy, context-aware encoding technology and coverage-driven pruning strategy, which enable *DiverFPS* to achieve high target AST-coverage with a small set of solutions.

To evaluate *DiverFPS*, we conducted extensive experiments on standard and publicly available benchmarks from SMT-LIB [2, 3], targeting the QF_FP (*i.e.*, quantifier-free formulas over floating-point) and QF_BVFP (*i.e.*, quantifier-free formulas over bit-vector and floating-point) logics. These benchmarks include SMT formulas derived from practical software testing and verification scenarios [2, 3] and have been widely studied [7, 13, 39, 49]. Our experimental results demonstrate that, compared to state-of-the-art SMT samplers, *DiverFPS* successfully achieves the target AST-coverage on a larger number of benchmarks, which existing samplers fail to reach and produces solution sets that are significantly smaller, with reductions of up to 92.9%. These results indicate improved effectiveness and efficiency in practical software testing and verification.

The main contributions of this work are summarized as follows.

- We introduce a novel SMT sampler, *DiverFPS*, targeting the theory of floating-point and its combination with bit-vectors, including the QF_FP logic and its extension QF_BVFP.
- We propose three novel techniques, solution-driven restart strategy, context-aware encoding technology, and coverage-driven pruning strategy, to enhance the performance of *DiverFPS*.
- Extensive experiments are conducted to demonstrate the effectiveness and superiority of *DiverFPS* over existing SMT samplers. The results show that *DiverFPS* achieves the target AST-coverage on a larger number of benchmarks while requiring significantly fewer solutions to reach the same target AST-coverage, thereby advancing the state of floating-point SMT sampling.

2 Preliminaries

This section provides the preliminaries of this work.

2.1 Basic Definitions and Notations for SMT Sampling

SMT Formula. An SMT formula is a logical expression defined over a set of variables whose semantics are constrained by background theories, which determine both the domain of the variables and the set of admissible operators. For an SMT formula φ , $V(\varphi)$ denotes the set of all variables

appearing in φ . In this work, we focus on the theories of bit-vectors and floating-point, which are described below.

- **Bit-vector.** In SMT formulas, bit-vector types are written as $(_ \text{ BitVec } k)$, where k denotes the width. A bit-vector of this type is a fixed-length binary sequence $(b_{k-1}, \dots, b_0)$ with each $b_i \in \{0, 1\}$, and the i -th least significant bit is denoted as $x(i)$. The bit-vector theory supports a wide range of operations, including bitwise operators (e.g., and, or, shift) and arithmetic operators (e.g., +, $\times$, <).
- **Floating-Point.** In SMT formulas, floating-point types are written as $(_ \text{ FloatingPoint } eb \ sb)$. A floating-point number of this type conforms to the IEEE 754 standard and is represented as a triple (s, e, m) , where $s \in \{0, 1\}$ is the single sign bit, $e \in \{0, 1\}^{eb}$ is the exponent field of width eb , and $m \in \{0, 1\}^{sb-1}$ is the significand of width $sb - 1$. The floating-point theory supports arithmetic operators (e.g., +, -, $\times$, $\div$), relational operators (e.g., <, $\leq$), and conversion operators between floating-point numbers and other theories (such as bit-vectors). The interplay of binary encoding and numerical semantics makes reasoning about floating-point constraints particularly challenging.

Assignment. Given an SMT formula φ with its set of variables $V(\varphi)$, an **assignment** σ for φ is a mapping from each variable in $V(\varphi)$ to a value in its corresponding domain. An assignment σ is a **solution** (or a satisfying assignment) if it makes the formula φ true (denoted $\sigma \models \varphi$). The set of all solutions for φ is denoted by $\text{Sol}(\varphi)$.

Abstract Syntax Tree (AST). The Abstract Syntax Tree of an SMT formula φ is a finite, rooted, ordered tree that represents its syntactic structure. In this tree, leaf nodes correspond to variables and constants, while internal nodes correspond to operators (e.g., logical connectives like $\wedge$, $\neg$ or theory functions like +, $\leq$). The children of an internal node are the ASTs of its operands, ordered according to the formula's structure.

AST Evaluation. Given an AST for an SMT formula φ and an assignment σ for its variables, the value of any node n in the tree, denoted $\sigma[[n]]$, is determined recursively. The process begins from the leaves: a leaf node for a constant c evaluates to c itself, while a leaf node for a variable v evaluates to its assigned value $\sigma(v)$. The value of an internal node with an operator op and children $c_1, \dots, c_k$ is then computed by applying the operation to the evaluated values of its children, as in $\sigma[[n]] = \text{op}(\sigma[[c_1]], \dots, \sigma[[c_k]])$.

AST-bit. An AST-bit is a tuple (b, v) , where b is a bit extracted from some AST node and $v \in \{0, 1\}$ is a bit value. Here, b can be a Boolean node or a specific bit of a bit-vector or floating-point node. For example, a bit-vector node x of type $(_ \text{ BitVec } 3)$ gives rise to six AST-bits: $(x(0), 0)$, $(x(0), 1)$, $(x(1), 0)$, $(x(1), 1)$, $(x(2), 0)$, $(x(2), 1)$. An AST-bit $(x(i), v)$ is said to be **covered** by an assignment σ if the i -th least significant bit of $\sigma[[x]]$ equals v . An AST-bit is considered **valid** if there exists at least one solution (i.e., satisfying assignment) that covers it; otherwise, it is **invalid**.

AST-coverage. Given an SMT formula φ , let $\mathcal{B}(\varphi)$ denote the set of all valid AST-bits derived from the AST of φ . For an assignment σ , the set of AST-bits covered by σ is defined as $C(\sigma) = \{(b, v) \in \mathcal{B}(\varphi) \mid \sigma \text{ covers } (b, v)\}$. For an assignment set S , the set of AST-bits covered by S is $C(S) = \bigcup_{\sigma \in S} C(\sigma)$. The **AST-coverage** of S is then defined as $\text{Cov}(S) = \frac{|C(S)|}{|\mathcal{B}(\varphi)|}$. As introduced in Section 1, AST-coverage is a standard metric in the SMT sampling domain for measuring the diversity of a solution set. A solution set with high AST-coverage triggers a wider range of behaviors in the software system, leading to the detection of more faults [17].

2.2 SMT Sampling Problem

As mentioned in Section 1, two key metrics are commonly employed to assess the quality of solution sets generated by SMT samplers: AST-coverage and solution set size. AST-coverage reflects the

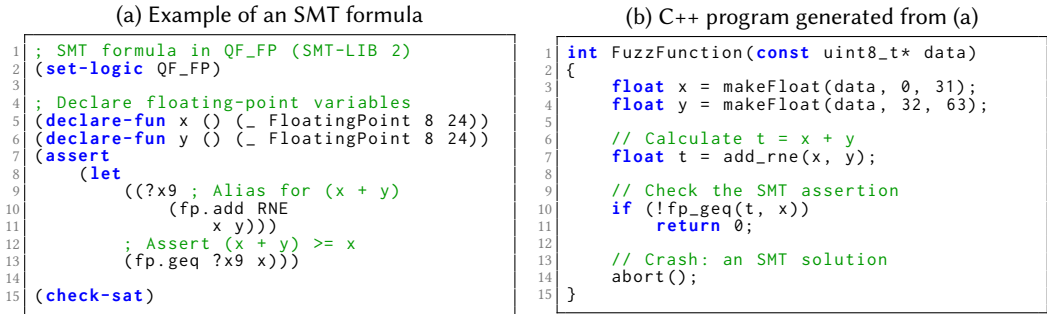


Fig. 1. Example of an SMT formula (a) and its corresponding C++ program (b).

fault detection capability [17], whereas the size of the solution set determines the efficiency of verification and testing in practice [12, 15, 26, 32, 51]. The importance of these two metrics has already been emphasized in a number of prior studies [7, 12, 15, 17, 26, 32, 51, 53]. Based on these metrics, the SMT sampling problem, which is the focus of this work is defined as follows:

DEFINITION 1. *Given an SMT formula Φ and a threshold $r \in (0, 1]$ specifying the target AST-coverage, the SMT Sampling problem requires finding a solution set S of Φ such that the AST-coverage of S is at least r i.e., $\text{Cov}(S) \geq r$, while $|S|$ is minimized.*

Solving the SMT sampling problem defined above generates a solution set that achieves high target AST-coverage, leading to high fault detection capability and small size, thereby enhancing both the effectiveness and efficiency of practical software verification and testing. Thus, effectively addressing this problem is of great importance.

2.3 SMT Solving via Coverage-Guided Fuzzing

As discussed in Section 1, handling floating-point constraints in SMT solving is particularly challenging. To address this difficulty, [39] proposed a coverage-guided fuzzing method for solving floating-point SMT formulas. A subsequent study [7] have further demonstrated that this method is also applicable to SMT sampling [7]. This subsection provides a brief overview of the underlying ideas and techniques.

2.3.1 Coverage-Guided Fuzzing. Coverage-guided fuzzing is a testing technique that explores program behaviors by iteratively generating and mutating inputs [38, 63]. The fuzzer monitors code coverage and retains inputs that trigger previously unseen execution paths, using them as seeds for further mutation. This coverage-driven process guides the exploration toward diverse and potentially fault-revealing behaviors. Consequently, the method has proven highly effective in automatically uncovering a wide range of software faults [39, 46]. Representative tools include AFL [63] and LibFuzzer [38].

2.3.2 SMT Solving via Coverage-Guided Fuzzing. As noted above, recent work has employed coverage-guided fuzzing for SMT solving [39]. In this method, an SMT formula is converted into an executable C++ program, with the formula's variables serving as program inputs; that is, each input corresponds directly to an assignment of the original SMT formula [7, 39]. The program is constructed such that it crashes if and only if all constraints of the original SMT formula are satisfied; hence, a crash signifies that the input corresponds to a solution (i.e., a satisfying assignment) [7, 39].

By utilizing a coverage-guided fuzzer, such as LibFuzzer [38], the input space can be systematically explored to identify such crashing inputs, thereby discovering solutions of the original SMT formula [7, 39]. The translation process from SMT formulas to C++ programs is described in detail in [39], and Figure 1 provides an illustrative example.

According to the results reported in [39], this method can handle floating-point constraints with high efficiency. Moreover, since the fuzzer is capable of rapidly generating a large number of inputs [7, 39], the method is naturally well-suited for SMT sampling scenarios [7].

3 Our Proposed *DiverFPS*

This section presents our proposed method, *DiverFPS*, an efficient floating-point SMT sampler.

3.1 Challenges and Potential Solutions

We begin with the challenges and potential solutions motivating *DiverFPS*.

3.1.1 Exploration Bottleneck. As discussed in Section 1, prior work [7] has applied coverage-guided fuzzing to floating-point SMT sampling and demonstrated strong performance [7]. In this approach, the SMT formula is translated into an executable C++ program such that solutions of the original SMT formula intentionally trigger program crashes; thus, crashes correspond to solutions of the original formula. Coverage-guided fuzzing generates candidate inputs by mutating existing seeds, enabling the rapid production of a large number solutions [7, 38, 39, 63]. However, the local and small-scale nature of these mutations, together with the bias introduced by the SMT samplers, which directs the fuzzer to use features of previously discovered solutions [7, 17], limits the sampler's ability to explore distant regions of the solution space. As a result, subsequent solutions yield progressively smaller incremental gains in diversity and AST-coverage, limiting the SMT sampler's ability to reach high target AST-coverage.

Potential Solution: To mitigate this bottleneck, *DiverFPS* introduces a solution-driven restart strategy. Unlike common coverage-guided restarts that trigger on code coverage stagnation [20, 58], a metric only loosely related to the difficulty of SMT solution discovery and AST-coverage, our solution-driven restart strategy redirects the fuzzing process only when the exploration for more diverse solutions becomes challenging. Additionally, *DiverFPS* employs a context-aware encoding technology that creates a tighter link between code coverage and AST-coverage, thereby guiding the fuzzer toward more meaningful directions. Together, these complementary strategies enable *DiverFPS* to mitigate the exploration bottleneck and better achieve the target AST-coverage.

3.1.2 Redundancy Issue. As discussed in Section 1, reducing the size of the solution set is crucial in practical software testing and verification. A smaller solution set reduces computational overhead and improves the efficiency of subsequent testing and analysis. However, due to the local and small-scale nature of mutations, new solutions initially differ only slightly from their seeds. As fuzzing progresses, differences among solutions gradually accumulate, yet a large portion of solutions remain similar. During this process, the existing SMT sampler generates numerous redundant solutions that cover the same or highly similar regions of the abstract syntax tree (AST). These redundant solutions contribute little or nothing to AST-coverage, causing the solution set to grow unnecessarily large without yielding commensurate gains in AST-coverage.

Potential Solution: To alleviate this issue, *DiverFPS* incorporates a pruning stage following the exploration phase. Specifically, it applies coverage-driven pruning strategy, which removes redundant solutions and retains only those contributing substantially to AST-coverage. This strategy effectively reduces the size of the solution set while preserving its ability to achieve the target AST-coverage, thereby improving the efficiency in software testing and verification.

3.2 Overall Design of *DiverFPS*

Algorithm 1: Overall Design of *DiverFPS*

Input: Φ : an SMT formula;
 r : the target AST-coverage;
Output: S : the set of generated solutions;

```

1   $U \leftarrow$  set of all AST-bits of  $\Phi$ ;
2  foreach  $b$  in  $U$  do
3     $W_h(b) \leftarrow 1$ ;
4   $P_0 \leftarrow$  executable C++ program translated from  $\Phi$ , where crashes correspond to solutions of  $\Phi$ ;
5   $P \leftarrow P_0, S \leftarrow \emptyset$ ;
6  while time budget not exhausted and  $\text{Cov}(S) < r$  do
7     $In \leftarrow$  candidate inputs for  $P$  generated by mutating seeds;
8     $Cr \leftarrow$  new crashes from  $In$ ;
9     $S_{new} \leftarrow$  solutions of  $\Phi$  extracted from crashes in  $Cr$ ;
10    $S \leftarrow S \cup S_{new}$ ;
11    $U \leftarrow U \setminus \{\text{AST-bits covered by } S_{new}\}$ ;
12   if Restart condition is met then
13      $P \leftarrow \text{ContextAwareEncode}(P_0, U)$ ;
14     foreach  $b$  in  $U$  do
15        $W_h(b) \leftarrow W_h(b) + 1$ ;
16     Reset fuzzer;
17  $S \leftarrow \text{Pruning}(\Phi, S, W_h)$ ;
18 return  $S$ ;
```

The overall design of *DiverFPS* is presented in Algorithm 1. As shown in Algorithm 1, given an input SMT formula Φ and a real number $r \in (0, 1]$ representing the target AST-coverage, *DiverFPS* produces a solution set S of Φ such that the AST-coverage of S is at least r i.e., $\text{Cov}(S) \geq r$, while minimizing its size $|S|$. The procedure consists of two stages: an exploration stage and a pruning stage. In the exploration stage (Line 1 – Line 16 in Algorithm 1), *DiverFPS* first translates the SMT formula Φ into an executable C++ program P_0 using the approach described in Section 2 and in [39] (Line 4 in Algorithm 1). In this transformation, crashes of P_0 correspond to solutions of Φ . Then, *DiverFPS* applies fuzzing to P_0 (Line 6 – Line 16 in Algorithm 1) and collects the solutions of Φ , i.e., the crashes of P_0 (Line 8 – Line 10 in Algorithm 1). During this stage, when exploring for more diverse solutions becomes challenging, *DiverFPS* employs solution-driven restart strategy (Line 12 – Line 16 in Algorithm 1) and context-aware encoding technology (Line 13 in Algorithm 1) to enhance the exploration of the solution space of Φ and mitigate the exploration bottleneck discussed in Section 3.1. In the subsequent pruning stage (Line 17 in Algorithm 1), *DiverFPS* removes redundant solutions from S , minimizing its size while ensuring that the target AST-coverage is maintained, thereby addressing the redundancy issue introduced in Section 3.1.

Through the overall design described above, *DiverFPS* can efficiently generate small solution sets that achieve the target AST-coverage for input SMT formulas. In the following, we provide a detailed description of each stage of *DiverFPS*, i.e., the exploration stage and the pruning stage.

3.3 Exploration Stage

In this subsection, we introduce the exploration stage of *DiverFPS*. As discussed in Section 3.1, applying coverage-guided fuzzing to floating-point SMT sampling can significantly improve efficiency.

However, due to the inherently local and small-scale nature of mutations, as well as the fact that existing SMT samplers tend to direct the fuzzer toward features of previously discovered solutions [7, 17], the sampler's ability to explore distant regions of the solution space is limited. Consequently, as fuzzing progresses, the solutions discovered provide diminishing improvements in AST-coverage. To address this exploration bottleneck, the exploration stage of *DiverFPS* combines the efficiency of coverage-guided fuzzing with two additional techniques that facilitate exploration of distant solution space regions. In particular, it adopts solution-driven restart strategy and context-aware encoding technology, which are detailed below.

3.3.1 Solution-Driven Restart Strategy. Restart is a well-established strategy in domains such as local search, the Boolean satisfiability problem (SAT), and combinatorial optimization [5, 31, 50, 62], where it is employed to prevent the search from being trapped in local optima. Similarly, in coverage-guided fuzzing, several restart strategies based on code coverage have also been proposed [20, 58]. The intuition is that, after a period of intensive exploration, the search trajectory often becomes biased toward previously visited regions, thereby limiting its ability to discover novel solutions.

In this work, we adopt the concept of restarts to design a solution-driven restart strategy that mitigates the exploration bottleneck in existing SMT sampling using coverage-guided fuzzing. Specifically, the restart condition is satisfied when the number of newly discovered solutions since the last restart reaches λ , a hyper-parameter of *DiverFPS* (Line 12 in Algorithm 1). This choice reflects the design intuition that, once a sufficient number of solutions have been collected since the last restart, the current search is likely to have largely fulfilled its potential. Unlike existing coverage-guided fuzzers, whose restart conditions rely primarily on code coverage [20, 58], our design better handles the challenges of SMT sampling, capturing both the difficulty of discovering solutions and the need for solution diversity and high AST-coverage. Triggering a restart at this point allows the fuzzing process to discard the current search state and resume with a newly generated set of random seeds (Line 16 in Algorithm 1), thereby redirecting the search toward previously unexplored regions of the solution space. This restart is further enhanced by context-aware encoding technology, which will be introduced later. Compared with standard coverage-guided fuzzing, which repeatedly mutates existing seeds within a limited neighborhood, our solution-driven restart strategy mitigates the diminishing gains in AST-coverage as fuzzing progresses. Consequently, the generated solution set is more likely to reach the target AST-coverage. The impact of this strategy on the overall performance of *DiverFPS* is evaluated in Section 5.

Algorithm 2: Context-aware Encoding Technology *i.e.*, ContextAwareEncode

Input: U : the set of uncovered AST-bits;

P_0 : the initial C++ program;

Output: P_{inst} : the newly instrumented C++ program;

```

1  $P \leftarrow P_0$ ;
2 foreach  $b$  in  $U$  do
3    $C_b \leftarrow$  a new code branch conditional on the value of  $b$ ;
4    $P \leftarrow P \cup \{C_b\}$ ;
5 return  $P$ ;
```

3.3.2 Context-Aware Encoding Technology. While the solution-driven restart strategy effectively forces the fuzzer to explore new regions of the solution space, the guidance within these new regions remains based on a static representation of the program. Existing fuzzers, including restart and mutation strategies, are almost entirely driven by code coverage, a metric only loosely related to

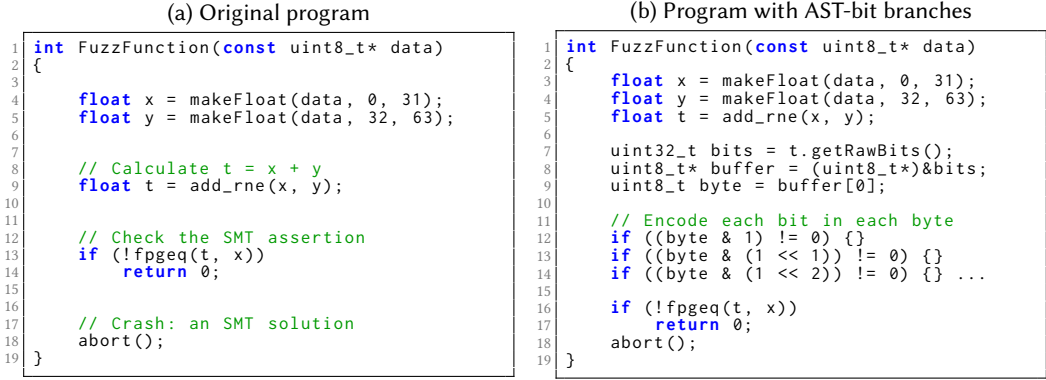


Fig. 2. Example of program instrumentation for AST-bits.

the difficulty of finding SMT solutions and AST-coverage. In Section 3.3.1, we mainly strengthened the connection between restarts and the challenge of discovering solutions; in the following, we introduce context-aware encoding technology to further tighten the link between AST-coverage and code coverage. A prior approach [7] instruments the program to guide the fuzzer toward covering a static, randomly selected subset of AST-bits. Specifically, it translates AST-bit coverage goals into a code coverage task by inserting new conditional branches, where each branch's execution depends on the corresponding AST-bit, as illustrated in Figure 2. The fuzzer, in its attempt to maximize code coverage, is thus naturally incentivized to generate inputs that explore these branches, *i.e.*, assignments for the SMT formula Φ that cover the corresponding AST-bits. A key limitation of this approach is its agnosticism toward individual AST-bits; it does not differentiate between covered and uncovered AST-bits, which may result in inefficient allocation of fuzzing effort. Consequently, code coverage obtained in this manner remains loosely coupled with AST-coverage. To address this, we propose context-aware encoding technology. The central idea is to utilize the information available at the moment a restart is triggered. We observe that the set of AST-bits remaining uncovered immediately before a restart can serve as an effective heuristic for prioritizing which AST-bits to target. This allows code coverage after each restart to dynamically and more directly reflect AST-coverage. *DiverFPS* leverages this observation to use the restart event not just for resetting the search, but also for directing the subsequent exploration phase.

This approach establishes a dynamic feedback loop that connects the state of the fuzzing process before a restart to the guidance for the process after it. When the restart condition is met (Line 12 in Algorithm 1), *DiverFPS* performs a context-aware encoding on the initial program P_0 (Line 13 in Algorithm 1). As detailed in Algorithm 2, this encoding process takes the set of currently uncovered AST-bits U , and the initial program P_0 as input. For each AST-bit in U (Line 2 in Algorithm 2), it generates new, semantic-preserving instrumentation code (Line 3 – Line 4 in Algorithm 2), following the approach illustrated in Figure 2. This encoding is context-aware because it is derived directly from the state of the preceding search. Finally, the fuzzer is restarted with this newly instrumented program (Line 16 in Algorithm 1). The fuzzer's intrinsic goal of maximizing code coverage in the context-aware encoded program P now naturally prioritizes inputs that exercise these new code paths, which in turn correspond to solutions covering the previously uncovered AST-bits, thus effectively increasing AST-coverage.

3.4 Pruning Stage

After the exploration stage, *DiverFPS* proceeds to the pruning stage. As discussed in Section 3.1, the solution set generated by coverage-guided fuzzing suffers from the redundancy issue. Since new solutions arise from small, iterative mutations on existing seeds, they often differ only slightly at first; while diversity gradually accumulates, this process naturally results in large clusters of superficially similar solutions. Thus, the pruning stage aims to mitigate this redundancy by selecting a minimal subset of solutions that preserves the target AST-coverage achieved.

Algorithm 3: Pruning Stage *i.e.*, Pruning

Input: Φ : an SMT formula;

S_c : a solution set of Φ ;

W_h : the set of hardness weights assigned to AST-bits of Φ ;

Output: S : a pruned solution set of Φ ;

```

1  $U \leftarrow$  set of all AST-bits of  $\Phi$ ,  $S^* \leftarrow \emptyset$ ;
2 while time budget not exhausted and  $S_c \neq \emptyset$  do
3    $S_r \leftarrow \emptyset$ ;
4   foreach  $s$  in  $S_c$  do
5      $score(s) \leftarrow 0$ ;
6     foreach  $b$  in  $U$  do
7       if  $s$  covers  $b$  then
8          $score(s) \leftarrow score(s) + W_h(b)$ ;
9     if  $score(s) = 0$  then
10       $S_r = S_r \cup \{s\}$ ;
11  $\alpha \leftarrow$  the solution that maximizes  $score$  in  $S_c$ ;
12  $S^* \leftarrow S^* \cup \{\alpha\}$ ,  $S_c \leftarrow S_c \setminus S_r$ ;
13  $U' \leftarrow \emptyset$ ;
14 foreach  $b$  in  $U$  do
15   if  $\alpha$  covers  $b$  then
16      $U' \leftarrow U' \cup \{b\}$ ;
17  $U \leftarrow U \setminus U'$ ;
18 return  $S^* \cup S_c$ ;
  
```

3.4.1 Iterative Pruning Process. To address the redundancy issue, *DiverFPS* incorporates a pruning stage that employs our proposed coverage-driven pruning strategy. The goal of the pruning stage is to iteratively construct a minimal solution set, while preserving the overall AST-coverage. This is achieved by repeatedly selecting the most impactful solution from a shrinking pool of candidates and then removing any solutions that become redundant as a result. As detailed in Algorithm 3, the pruning stage is an anytime, iterative optimization process. It begins with the full set of solutions from the exploration stage serving as the initial candidate set S_c , while the core selected set S^* starts empty (Line 1 in Algorithm 3). The pruning stage then proceeds iteratively as follows: In each iteration, *DiverFPS* first calculates a metric we formally define as the impact score for every solution remaining in the candidate set S_c . This impact score measures a solution's ability to cover AST-bits not yet covered by the solutions already selected into S^* (Line 4 – Line 10 in Algorithm 3). The solution with the highest impact score is then identified as the most impactful one for the current step (Line 11 in Algorithm 3). The detailed calculation of the impact score is presented

later in this section. Then, this identified solution α is selected and moved from S_c to S^* (Line 12 in Algorithm 3). Following this selection, the set of covered AST-bits is updated (Line 13 – Line 17 in Algorithm 3). As a result, any solutions in the candidate set S_c that have been rendered redundant (i.e., they no longer cover any remaining uncovered bits) will be removed at the beginning of the subsequent iteration (Line 9 – Line 10 in Algorithm 3). This loop continues, progressively building the S^* set with high-impact solutions while efficiently shrinking the candidate pool S_c . Intuitively, in the loop, *DiverFPS* selects in each iteration the most impactful solution α . This solution not only substantially increases the AST-coverage when added to S^* , but also enables extensive pruning of S_c based on its impact. Thus, the pruning process effectively reduces the size of the solution set.

In addition, as an anytime algorithm, this process can be stopped at any point. To guarantee that the total AST-coverage is preserved regardless of when the algorithm terminates, the final returned set is the union of the core selected set and the remaining candidates i.e., $S^* \cup S_c$. The S^* set contains the identified high-impact, non-redundant solutions, while the remaining S_c serve as a safety net to cover any bits not yet accounted for by the core set. As the algorithm runs longer, the size of this returned union progressively decreases, converging toward the minimal solution set.

3.4.2 Scoring Heuristics. Here we detail the calculation of the impact score mentioned above. As previously described, the pruning stage iteratively selects the most impactful solution to build a minimal set. Therefore, the definition of the impact score, which guides this selection, is of critical importance. A straightforward approach would be to define a solution's impact score as simply the number of AST-bits in the current set U that it can cover. While intuitive, this method suffers from a limitation: it treats all AST-bits as equally important. However, we observe that the difficulty of covering different AST-bits varies significantly. Certain AST-bits are covered by the majority of possible solutions, while others can be covered by only a small subset of specific solutions. As prioritizing more challenging elements has been applied in several related domains [4, 42, 47], e.g., the SAT problem and the Constraint satisfaction problem (CSP), which are closely related to SMT sampling, a more effective strategy should prioritize solutions that cover 'hard-to-cover' AST-bits.

To quantify the difficulty of covering each AST-bit, we leverage the heuristic insights from the exploration stage. As discussed in Section 3.3, a restart event indicates that the search has likely encountered a bottleneck. Thus, the set of currently uncovered AST-bits serves as an indicator of difficulty, as these bits represent parts of the solution space that have not yet been explored and may require additional targeted efforts to cover. Based on this, we define a hardness weight (i.e., W_h) for each AST-bit. These hardness weights are initialized to 1 (Line 2 – Line 3 in Algorithm 1). During the exploration stage, every time a restart is triggered, the hardness weight of each currently uncovered AST-bit is incremented by one (Line 14 – 15 in Algorithm 1). This ensures that AST-bits repeatedly resisting coverage accumulate a high hardness weight, which in turn guides the pruning stage to prioritize covering these difficult AST-bits when optimizing the solution set size, thereby ensuring that the most challenging parts are adequately addressed.

With this weighting scheme established, we can now formally define the impact score. The impact score of a candidate solution s is the sum of the hardness weights of the currently AST-bits in U that s covers (Line 4 – Line 10 in Algorithm 3). By defining the impact score in this manner, *DiverFPS* ensures the selection process is driven not merely by the quantity of new coverage, but more importantly, by the quality and difficulty of that coverage. This prioritizes solutions that are most essential for achieving a complete and diverse representation of the solution space.

In summary, coverage-driven pruning strategy progressively eliminates redundant solutions from the exploration-stage set, effectively addressing the redundancy issue.

3.4.3 Efficiency of the Pruning Stage and Its Time Allocation. As discussed above, the pruning stage is lightweight and does not require expensive SMT solving. It computes impact scores in linear

time and then greedily retains high-scoring solutions while removing redundant ones with zero impact. Since greedy heuristics for test suite reduction are widely recognized as highly efficient [40], *DiverFPS*, as an anytime algorithm, only needs to allocate a small fraction of the overall time budget to pruning. Accordingly, *DiverFPS* switches to pruning when either (1) the target AST-coverage has been achieved, allowing an early start, or (2) 10 seconds of the time budget remain, triggering a forced start. The rationale behind the choice of the 10-second threshold is analyzed in Section 6.1.

4 Experimental Design

4.1 Public Benchmarks and Competitors

We use a set of public benchmarks from SMT-LIB [2, 3], comprising 160 QF_FP benchmarks and 702 QF_BVFP benchmarks (862 in total). These benchmarks include SMT formulas derived from practical software testing and verification scenarios, such as symbolic execution, bounded model checking, and static analysis[2, 3] and have been widely studied [7, 13, 39, 49]. The state-of-the-art floating-point SMT sampler [7] also evaluates its performance on the exact same benchmark set. All adopted benchmarks are publicly available in our repository¹.

We compare *DiverFPS* against the following state-of-the-art SMT samplers in our experiments.

JFSampler [7] represents the state-of-the-art SMT sampler for the floating-point theory, with support for the QF_FP and QF_BVFP logics. Experimental results reported in [7] demonstrate that *JFSampler* is highly effective in achieving high AST-coverage.

SMTSampler [17] is an SMT sampler with a primary focus on bit-vector logics. According to [17], it performs well in terms of achieving high AST-coverage. Although *SMTSampler* does not natively support the floating-point theory, the authors of *JFSampler* extended it to handle the QF_FP and QF_BVFP logics for their experiments [7]. We therefore include this extended version of *SMTSampler* in our comparison.

The implementations of *JFSampler*² and *SMTSampler*³ are both publicly available online.

4.2 Research Questions

Our experiments aim to answer the following research questions (RQs).

RQ1: Is *DiverFPS* more effective than its competitors at achieving target AST-coverage?

In this RQ, we compare *DiverFPS* with its competitors by the number of benchmarks on which the target AST-coverage is achieved, across multiple AST-coverage thresholds.

RQ2: Can *DiverFPS* produce smaller solution sets for the same AST-coverage?

In this RQ, we compare the solution set sizes generated by *DiverFPS* and the state-of-the-art competitors on benchmarks where the target AST-coverage is achieved.

RQ3: Does *DiverFPS* statistically significantly outperform its competitors?

In this RQ, we conducted significance testing and effect size analysis to assess whether the performance improvements of *DiverFPS* over its competitors are statistically significant.

RQ4: How does the performance of *DiverFPS* evolve over time, given its anytime nature?

In this RQ, we analyze *DiverFPS* performance across varying cutoff times.

RQ5: Do the core techniques proposed in this work contribute to the performance improvements of *DiverFPS*?

In this RQ, we conduct an ablation study to assess the contribution of each core technique.

RQ6: How does the setting of the hyper-parameter affect the performance of *DiverFPS*?

In this RQ, we study the impact of *DiverFPS*'s hyper-parameters on performance.

¹<https://github.com/ShuangyuLyu/DiverFPS>

²<https://github.com/srg-imperial/jfs-sampling>

³<https://github.com/srg-imperial/jfsampler-SMTSampler>

4.3 Experimental Setup

All experiments were conducted on a computing machine equipped with an Intel(R) Xeon(R) Gold 6130 CPU operating at 2.10GHz, 1TB of RAM, and running the Ubuntu 22.04.5 LTS operating system. For our *DiverFPS*, the hyper-parameter λ was set to 100, and for its competitors, we adopted the default configurations as recommended by their respective authors.

Since *DiverFPS* and its competitors (*i.e.*, *JFSampler* and *SMTSampler*) are all randomized algorithms, we performed 10 independent runs for each sampler on every benchmark, following the experimental setup in [7]. Moreover, *DiverFPS* and its competitors are anytime algorithms, producing a solution set at any point, striving to achieve the target AST-coverage (though it may not be reached). We set a timeout of 5 minutes per run, consistent with [7], and limited the maximum solution set size to 1,000, following [43, 44]. Building on this setup, we evaluated the samplers under six target AST-coverage levels: 80%, 90%, 95%, 98%, 99%, and 99.5%, denoted as ‘Cov’. Since computing the exact AST-coverage is computationally expensive [7, 53], consistent with prior work [53], AST-coverage was normalized by computing the ratio of covered AST-bits to the total number of AST-bits covered by at least one sampler in the evaluation. For constructing ASTs, we adopted the approaches in [7, 17, 53], employing the widely used Z3 solver [16] as the standard.

Under our experimental setup, we evaluate each sampler’s performance for each target AST-coverage level. We computed ‘#suc’, defined as the number of benchmarks on which the target AST-coverage was successfully achieved, averaged across 10 independent runs. Additionally, for runs that successfully achieved the target AST-coverage, we report ‘#sol’, the average minimum number of solutions required to achieve the target AST-coverage. To provide context, we also report ‘#suc.all’, the number of benchmarks where all competing samplers achieved the target AST-coverage in all 10 runs, and ‘#suc.any’, the number where at least one competing sampler was able to do so in any run. In our comparison, the best-performing sampler is the one with the highest ‘#suc’. Any ties are broken by giving preference to the sampler with the lowest ‘#sol’. Results determined to be the best by these criteria are highlighted in **boldface**.

5 Experimental Results

5.1 RQ1: Effectiveness in Achieving Target AST-Coverage

We compare *DiverFPS* against *JFSampler* and *SMTSampler* on the set of 862 public benchmarks, comprising 160 QF_FP benchmarks and 702 QF_BVFP benchmarks. Due to space constraints, we do not report the complete results here. The full experimental results of *DiverFPS* and its competitors on all benchmarks are publicly available in our repository¹. To provide an overview, we summarize the ‘#suc’ and ‘#sol’ metrics across all benchmarks in Table 1.

As shown in Table 1, *DiverFPS* consistently achieves the best performance among the three samplers. Specifically, on both QF_FP and QF_BVFP benchmarks, *DiverFPS* succeeds in achieving AST-coverage on the highest average number of benchmarks across all target AST-coverage levels (*i.e.*, achieves the highest ‘#suc’). In particular, under the most stringent target of 99.5% AST-coverage, *DiverFPS* successfully generates solution sets for an average of 58.5 QF_FP benchmarks and 630.2 QF_BVFP benchmarks over 10 independent runs. In comparison, *JFSampler* achieves the 99.5% target AST-coverage on an average of only 24.9 QF_FP benchmarks and 261.1 QF_BVFP benchmarks, while *SMTSampler* succeeds on an average of 5.3 QF_FP benchmarks and 603.6 QF_BVFP benchmarks. These results clearly demonstrate that *DiverFPS* is much more effective than its competitors (*i.e.*, *JFSampler* and *SMTSampler*) in attaining the target AST-coverage.

Moreover, the results indicate that *SMTSampler*, which is designed for bit-vector SMT sampling, performs much better on QF_BVFP than on QF_FP benchmarks, highlighting that SMT sampling strategies tailored for the bit-vector theory do not directly apply to floating-point theory.

Table 1. Results of the ‘#suc’ and ‘#sol’ metrics obtained by *DiverFPS* and its state-of-the-art competitors.

Logic	Cov	<i>DiverFPS</i>		<i>JFSampler</i>		<i>SMTSampler</i>	
		#suc	#sol	#suc	#sol	#suc	#sol
QF_FP	80.0%	104.1	2.9	90.0	39.3	15.1	40.4
	90.0%	89.6	3.3	76.6	33.0	12.3	81.9
	95.0%	82.1	4.0	67.7	85.1	10.3	104.0
	98.0%	75.7	4.9	43.5	104.5	7.0	62.5
	99.0%	68.4	5.5	32.6	114.8	5.7	37.6
	99.5%	58.5	5.8	24.9	104.5	5.3	25.5
QF_BVFP	80.0%	682.1	2.1	673.4	10.9	624.4	38.2
	90.0%	676.5	2.3	648.2	19.9	623.0	72.7
	95.0%	673.4	2.6	618.4	32.1	620.8	94.0
	98.0%	662.0	3.2	573.6	33.4	612.1	105.3
	99.0%	645.8	3.2	553.0	34.1	607.5	108.6
	99.5%	630.2	3.5	261.1	56.1	603.6	110.3

5.2 RQ2: Solution Set Size Comparison

Table 2. Results of the ‘#suc’ metric for *DiverFPS* and its state-of-the-art competitors on the benchmarks that were successfully covered by all samplers across all 10 independent runs.

Logic	Cov	#suc.all	#sol		
			<i>DiverFPS</i>	<i>JFSampler</i>	<i>SMTSampler</i>
QF_FP	80.0%	61	2.1	19.8	--
	90.0%	55	2.8	17.6	--
	95.0%	40	3.4	47.7	--
	98.0%	23	3.9	23.4	--
	99.0%	18	4.3	42.2	--
	99.5%	14	4.1	21.3	--
QF_BVFP	80.0%	602	2.0	6.9	38.3
	90.0%	578	2.1	14.4	69.2
	95.0%	536	2.3	16.0	71.8
	98.0%	508	2.6	16.2	66.7
	99.0%	501	2.7	20.8	65.4
	99.5%	189	2.9	17.2	67.3

In this subsection, we compare the solution set sizes generated by *DiverFPS* and its competitors. Based on the results of RQ1, *SMTSampler* achieves the target AST-coverage on only a few QF_FP benchmarks (approximately 5 – 15 out of 160); therefore, it is excluded from the comparison on QF_FP benchmarks. Table 2 reports the average solution set size required by each sampler to reach various target AST-coverage levels. The statistics are computed only on benchmarks where all considered samplers (for QF_FP benchmarks, *DiverFPS* and *JFSampler*; for QF_BVFP benchmarks,

DiverFPS, *JFSampler*, and *SMTSampler*) successfully attained the corresponding target AST-coverage across all 10 independent runs.

As shown in Table 2, *DiverFPS* consistently produces substantially smaller solution sets across all target AST-coverage levels. On QF_FP benchmarks, *DiverFPS* reduces the solution set size by at least 80.8% at 99.5% target AST-coverage, and up to 92.9% at 95.0% target AST-coverage, compared to *JFSampler*. On QF_BVFP benchmarks, *DiverFPS* achieves reductions of 71.0% to 87.0% relative to the second-best performing sampler. These results indicate that *DiverFPS* effectively alleviates the redundancy issue in floating-point SMT sampling. As discussed in Section 1 such reductions can considerably lower the time and cost of testing in practical scenarios.

5.3 RQ3: Statistical Significance of Performance Differences

Table 3. Number of benchmarks where *DiverFPS* shows statistically significant.

Logic	Competing Samplers	#win.sig / #suc.any					
		80.0%	90.0%	95.0%	98.0%	99.0%	99.5%
QF_FP	<i>DiverFPS</i> v.s. <i>JFSampler</i>	79/117	80/110	78/100	73/88	67/80	57/71
	<i>DiverFPS</i> v.s. <i>SMTSampler</i>	106/118	86/110	79/99	73/89	67/81	57/72
QF_BVFP	<i>DiverFPS</i> v.s. <i>JFSampler</i>	476/687	673/687	671/683	663/678	648/667	634/651
	<i>DiverFPS</i> v.s. <i>SMTSampler</i>	678/689	673/689	671/687	659/682	643/672	623/664

To investigate whether the performance advantages of *DiverFPS* over its competitors *JFSampler* and *SMTSampler*, as observed in RQ1 and RQ2, are statistically significant, we conduct the Mann-Whitney U test on the results of 10 independent runs for each benchmark. In addition, we compute the Vargha–Delaney effect sizes for each pairwise comparison. Specifically, for each benchmark we conduct pairwise comparisons between *DiverFPS* and each competitor (i.e., *JFSampler* and *SMTSampler*). We require that (1) the p-value of the Mann–Whitney U test at the 95% confidence level is below 0.05, and (2) the Vargha–Delaney effect size is greater than 0.71, which is commonly regarded as indicating a large effect size [44, 45, 57, 61]. If these criteria are satisfied, we conclude that *DiverFPS* achieves a statistically significant improvement over the corresponding competitor on this benchmark. Notably, for any target AST-coverage level, significance testing is meaningless on benchmarks where both competing samplers fail to reach the target coverage in all 10 independent runs. Therefore, the test is conducted only on benchmarks where at least one of the two competing samplers reaches the target coverage in at least one run.

Table 3 summarizes the number of benchmarks under each target AST-coverage where *DiverFPS* achieves statistically significant superiority (denoted as ‘#win.sig’), with the number of benchmarks considered for the significance test (‘#suc.any’) provided as a reference. Complete results for all benchmarks are available in our repository¹. According to Table 3, *DiverFPS* significantly outperforms its competitors on the majority of benchmarks. In particular, it achieves statistically significant improvements on at least 67.5% of the benchmarks, and up to 98.4% in the best case.

5.4 RQ4: Impact of Different Cutoff Times

As discussed in Section 4, *DiverFPS* and its competitors, *JFSampler* and *SMTSampler*, are anytime algorithms. They can terminate at any point, producing a solution set that strives to achieve the target AST-coverage (though it may not always be reached). In the other RQs, we followed the experimental setup of *JFSampler* [7], fixing the cutoff time to 5 minutes. In this subsection, we

Table 4. Results of the ‘#suc’ and ‘#sol’ metrics obtained by *DiverFPS* at different cutoff times.

Logic	Cov	<i>DiverFPS-1min</i>		<i>DiverFPS-5min</i>		<i>DiverFPS-10min</i>	
		#suc	#sol	#suc	#sol	#suc	#sol
QF_FP	80.0%	100.9	2.7	104.1	2.9	105.0	2.7
	90.0%	86.1	3.5	89.6	3.3	91.3	3.4
	95.0%	79.8	4.3	82.1	4.0	83.7	4.1
	98.0%	65.7	5.2	75.7	4.9	75.7	5.0
	99.0%	55.2	5.4	68.4	5.5	71.1	5.6
	99.5%	47.3	5.7	58.5	5.8	62.1	6.0
QF_BVFP	80.0%	680.4	2.1	682.1	2.1	682.4	2.0
	90.0%	669.6	2.3	676.5	2.3	677.5	2.3
	95.0%	664.7	2.7	673.4	2.6	675.0	2.6
	98.0%	639.7	3.0	662.0	3.2	665.7	3.1
	99.0%	631.1	3.3	645.8	3.2	651.5	3.4
	99.5%	615.8	3.5	630.2	3.5	632.3	3.5

instead evaluate the impact of different cutoff times on the performance of *DiverFPS*. Specifically, we conducted experiments under three cutoff times, *i.e.*, 1, 5, and 10 minutes. Due to space constraints, Table 4 presents only the overall results, while the complete results are provided in our repository¹. In Table 4, We denote the variants of *DiverFPS* under different cutoff times as *DiverFPS-1min*, *DiverFPS-5min*, and *DiverFPS-10min*, corresponding to cutoff times of 1, 5, and 10 minutes, respectively.

According to Table 4, across different target AST-coverage, *DiverFPS* with longer cutoff times can generate solution sets that achieve the target AST-coverage for more benchmarks, indicating that additional time enables *DiverFPS* to explore the solution space more effectively. It is also noteworthy that even under the shortest cutoff time of 1 minute, *DiverFPS* already outperforms both *JFSampler* and *SMTSampler*, further highlighting its robustness and efficiency.

5.5 RQ5: Ablation Study of Core Techniques

As introduced in Section 3, *DiverFPS* incorporates three core techniques: solution-driven restart strategy, context-aware encoding technology, and coverage-driven pruning strategy. To assess the impact of each core technique, we developed four alternative versions of *DiverFPS*, denoted as *Alt-1*, *Alt-2*, *Alt-3*, and *Alt-4*. Specifically, *Alt-1* disables solution-driven restart strategy, *Alt-2* removes context-aware encoding technology, *Alt-3* replaces coverage-driven pruning strategy with a naive strategy that directly traverses the solution set produced in the exploration stage and discards solutions that do not contribute to AST-coverage, and *Alt-4* removes the scoring heuristics while retaining the coverage-driven pruning framework. The complete results are available in our repository¹, and Table 5 summarizes the overall performance.

From Table 5, *DiverFPS* achieves better overall performance than all its alternative versions. Compared with *Alt-1* and *Alt-2*, *DiverFPS* tends to generate solution sets that reach the target AST-coverage on more benchmarks. Relative to *Alt-3* and *Alt-4*, *DiverFPS* generates markedly smaller solution sets. Thus, each core technique is shown to contribute to *DiverFPS*’s overall performance.

5.6 RQ6: Impact of Hyper-Parameter Settings

As introduced in Section 3, *DiverFPS* has a single hyper-parameter, λ , which determines the restart condition in the exploration stage. *DiverFPS* triggers a restart when the number of newly discovered

Table 5. Results of the ‘#suc’ and ‘#sol’ metrics obtained by *DiverFPS* and its alternative versions.

Logic	Cov	<i>DiverFPS</i>		<i>Alt-1</i>		<i>Alt-2</i>		<i>Alt-3</i>		<i>Alt-4</i>	
		#suc	#sol	#suc	#sol	#suc	#sol	#suc	#sol	#suc	#sol
QF_FP	80.0%	104.1	2.9	89.0	5.0	100.9	2.9	103.3	25.3	99.5	8.1
	90.0%	89.6	3.3	74.4	4.9	84.0	3.2	88.7	27.4	84.7	8.7
	95.0%	82.1	4.0	63.5	6.4	78.1	3.9	81.4	33.5	79.1	11.7
	98.0%	75.7	4.9	41.2	7.0	72.1	4.9	74.4	41.8	72.9	15.1
	99.0%	68.4	5.5	31.1	6.2	65.5	5.5	68.4	43.1	67.0	17.7
	99.5%	58.5	5.8	24.2	5.8	57.4	5.7	58.2	41.5	58.5	18.2
QF_BVFP	80.0%	682.1	2.1	675.0	2.5	682.1	2.1	681.7	8.6	680.9	3.4
	90.0%	676.5	2.3	649.7	3.0	673.8	2.3	675.6	13.2	675.7	6.5
	95.0%	673.4	2.6	620.4	3.6	669.3	2.6	672.7	17.7	672.9	10.3
	98.0%	662.0	3.2	573.3	3.8	655.5	3.1	661.0	20.2	661.8	13.3
	99.0%	645.8	3.2	552.7	3.9	636.7	3.1	644.1	21.2	645.6	14.0
	99.5%	630.2	3.5	324.8	4.3	556.8	3.6	629.1	21.5	628.3	13.7

Table 6. Results of the ‘#suc’ and ‘#sol’ metrics obtained by *DiverFPS* with various settings of λ .

Logic	Cov	$\lambda = 10$		$\lambda = 100$		$\lambda = 1,000$		$\lambda = 10,000$	
		#suc	#sol	#suc	#sol	#suc	#sol	#suc	#sol
QF_FP	80.0%	100.2	2.8	104.1	2.9	102.3	2.8	102.2	2.9
	90.0%	86.0	3.5	89.6	3.3	88.1	3.3	87.3	3.3
	95.0%	77.7	4.0	82.1	4.0	80.6	3.9	80.3	4.0
	98.0%	72.5	4.9	75.7	4.9	73.8	4.8	73.6	4.8
	99.0%	67.1	5.6	68.4	5.5	66.9	5.4	66.5	5.5
	99.5%	55.3	5.7	58.5	5.8	58.2	5.7	57.8	5.8
QF_BVFP	80.0%	681.4	2.0	682.1	2.1	681.6	2.0	681.6	2.0
	90.0%	676.8	2.3	676.5	2.3	675.7	2.3	675.8	2.3
	95.0%	671.1	2.5	673.4	2.6	671.5	2.6	672.0	2.6
	98.0%	660.8	2.9	662.0	3.2	660.2	3.1	660.8	3.1
	99.0%	645.8	3.1	645.8	3.2	643.7	3.2	644.4	3.2
	99.5%	568.6	3.4	630.2	3.5	598.5	3.6	598.6	3.6

solutions since the last restart reaches λ . A smaller λ results in more frequent restarts, while a larger λ reduces the frequency of restarts. We evaluated the performance of *DiverFPS* under four different λ settings (i.e., $\lambda = 10$, $\lambda = 100$, $\lambda = 1,000$ and $\lambda = 10,000$). The complete experimental results are available in our repository¹, and Table 6 summarizes the overall performance. From Table 6, *DiverFPS* demonstrates strong performance across all λ settings. In particular, $\lambda = 100$ yields the best overall performance, and we therefore adopt $\lambda = 100$ as the default setting.

6 Discussion

In this section, we analyze the reserved time for pruning, evaluate *DiverFPS* in real-world embedded software testing, and discuss threats to validity.

6.1 Sensitivity Analysis of the Reserved Time for Pruning

Table 7. Results of the ‘#suc’ and ‘#sol’ metrics obtained by *DiverFPS* at different reserved times for pruning.

Cov	<i>DiverFPS-pur-5s</i>		<i>DiverFPS-pur-10s</i>		<i>DiverFPS-pur-20s</i>		<i>DiverFPS-pur-30s</i>	
	#suc	#sol	#suc	#sol	#suc	#sol	#suc	#sol
80.0%	783.0	2.2	786.2	2.2	784.6	2.2	784.4	2.2
90.0%	763.8	2.4	766.1	2.4	764.7	2.4	764.7	2.4
95.0%	755.1	2.8	755.5	2.8	756.2	2.7	756.0	2.7
98.0%	738.8	3.3	737.7	3.4	739.0	3.4	739.2	3.4
99.0%	719.1	3.5	714.2	3.5	719.2	3.5	719.2	3.5
99.5%	688.7	3.7	688.7	3.7	688.1	3.7	688.4	3.7

As discussed in Section 3.4, *DiverFPS*, needs to reserve a short amount of time within the overall time budget for the pruning stage. In this subsection, we conduct a sensitivity analysis on the amount of time reserved for pruning. Specifically, we evaluate four variants of *DiverFPS* that reserve 5s, 10s, 20s, and 30s for the pruning stage, denoted by *DiverFPS-pur-5s*, *DiverFPS-pur-10s*, *DiverFPS-pur-20s*, and *DiverFPS-pur-30s*, respectively. All other experimental settings are the same as those described in Section 4. The results are summarized in Table 7.

According to Table 7, the differences among these settings are negligible. For example, when the target AST-coverage is 99.5%, the average numbers of successful benchmarks (‘#suc’) are 688.7, 688.7, 688.1, and 688.4, respectively, while the corresponding average solution set sizes (‘#sol’) are all 3.7. Similar observations also hold for the other target AST-coverage settings. These results confirm that reserving 10 seconds for the pruning stage is sufficient.

6.2 Effectiveness in Real-World Embedded Software Testing

Table 8. Results of the code coverage achieved by *DiverFPS* and its competitors on the 7 systems.

Systems	<i>DiverFPS</i>	<i>JFSampler</i>	<i>SMTSampler</i>
AFC	90.9%	84.5%	0.0%
AP12B	68.6%	54.0%	0.0%
Regs12B	80.8%	62.3%	0.0%
TankController	92.5%	75.0%	91.7%
TCS_1	90.5%	76.8%	59.1%
TCS_2	99.2%	80.8%	34.2%
Tustin Integrator	100.0%	80.0%	0.0%
average	88.9%	73.3%	26.4%

As mentioned in Section 1, SMT sampling can benefit many application domains, such as software testing and symbolic execution. To demonstrate its practical usefulness, we additionally conducted a case study on seven real-world embedded systems modeled in Simulink from the industrial, automotive, and aerospace domains. These systems include standard benchmarks such as the Traction Control System (TCS) and the Engine Air-Fuel Control (AFC) system, as well as other embedded systems from [33] and [60]. We encoded these embedded systems into SMT formulas

using *CBMC* [37], and then employed *DiverFPS* and its competitors to sample solutions over them. To allow the samplers to explore the solution space as thoroughly as possible, the target AST-coverage was set to 100%. We performed 10 independent runs, while keeping the other settings, such as the computing environment and cutoff time, consistent with those described in Section 4. The experimental results are summarized in Table 8.

According to Table 8, the results show that *DiverFPS* achieved the highest code coverage across all systems. Overall, across all 7 embedded systems, *DiverFPS* reached an average code coverage of 88.9%, compared with 73.3% for *JFSampler* and 26.4% for *SMTSampler*. This demonstrates that *DiverFPS* is more effective at improving code coverage in real-world testing tasks.

6.3 Threats to Validity

6.3.1 Representativeness of the Benchmarks. As introduced in Section 4, the benchmarks used in this work are drawn from SMT-LIB [2, 3], a widely adopted and standard source for SMT-related research, including SMT sampling [7, 13, 17, 18, 39, 49, 59]. The specific set of 862 benchmarks we sampled has also been used in prior studies on floating-point SMT [7, 39], and the main baseline sampler we compare against, *JFSampler* [7], uses the same 862 benchmarks [7]. Furthermore, these benchmarks include SMT formulas derived from practical software testing and verification scenarios, ensuring their relevance to real-world applications [2, 3].

6.3.2 Randomness of the Algorithms. All samplers evaluated in our experiments, including *DiverFPS* and its competitors *JFSampler* [7] and *SMTSampler* [17], are randomized, making the results of a single run per benchmark insufficient for a reliable performance assessment. Following prior studies [7, 44, 45], each sampler was executed 10 times on every benchmark to account for randomness. In addition, we performed significance tests and calculated effect sizes to rigorously analyze the results, thereby mitigating the potential impact of algorithmic randomness.

6.3.3 Anytime Samplers and Cutoff. All samplers evaluated in our experiments are anytime methods: they can produce a solution set at any point during execution and aim to achieve the target AST-coverage, though not always successfully. To ensure fairness and comparability, all samplers were assigned the same cutoff time, which matches the settings used in prior studies [7]. Additionally, we analyzed the performance of *DiverFPS* under different cutoff times. The results indicate that it performs consistently well across all tested settings.

7 Related Work

Satisfiability modulo theories (SMT) plays a central role in the testing and verification of software and hardware systems [6, 14, 22, 23, 25, 27, 54]. Among various theories, floating-point is significant due to its broad applications [13, 21, 34]. In many of these applications, SMT formulas are encoded from real-world software systems, and they require not just one but a diverse set of solutions [7, 17, 53]. This motivates our focus on floating-point SMT sampling. However, despite its importance, research on SMT sampling, especially for the floating-point theory, remains limited.

Compared with SMT sampling, the simpler setting of Boolean Satisfiability (SAT) sampling has attracted considerable attention, and many approaches have been proposed in that context. For instance, existing techniques include those leveraging Markov Chain Monte Carlo (MCMC) [35, 36], universal hashing [8, 11], and model counting [9, 10], alongside approaches that adapt solver heuristics [24, 48] or utilize epoch-based methods [19].

For SMT sampling, several samplers have been developed with a focus on the bit-vector theory, which is closely related to SAT [17, 18, 59]. Examples include *csb* [59], which eagerly converts SMT formulas into SAT, and mutation-based approaches at the bit level [17, 18]. However, floating-point formulas cannot be as easily reduced to SAT as bit-vectors, and mutation at the bit level requires

frequent solver invocations. Since handling floating-point constraints presents great challenges [7, 39], these strategies do not extend well to floating-point sampling, as confirmed by both prior studies [7] and our experiments in Section 5. Similarly, prior SMT sampling work on integers [53] relies on properties unique to integers, and thus cannot be applied to floating-point constraints.

Given these issues, recent work [7] uses coverage-guided fuzzing for floating-point SMT sampling. Coverage-guided fuzzing is a testing technique that explores program behavior and aims to expose crashes by maximizing code coverage [38, 63]. By translating an SMT formula into an executable C++ program, where assignments to SMT variables correspond to program inputs and solutions to the formula are treated as program crashes, coverage-guided fuzzers can be used to efficiently generate large numbers of solutions. This approach has shown promising results in floating-point SMT sampling [7]. Nevertheless, this sampler still faces two major challenges. First, the exploration bottleneck, where achieving high target AST-coverage is difficult, limits the effective exploration of various behaviors in practical software testing and reduces the method's ability to uncover faults. Second, the redundancy issue, in which large solution sets contain duplicate or uninformative solutions, leading to redundant test executions and reduced testing efficiency in practice.

To address these challenges, we propose *DiverFPS*, a novel floating-point SMT sampler that mitigates both exploration bottleneck and redundancy issue. *DiverFPS* can effectively generate small solution sets that achieve high target AST-coverage for SMT formulas in QF_FP and QF_BVFP logics. Experimental results show that *DiverFPS* not only achieves the target AST-coverage on more benchmarks than existing SMT samplers, but also produces significantly smaller solution sets at the same AST-coverage target. In practice, the former improves the effectiveness of software testing and verification, while the latter enhances efficiency.

8 Conclusion

In this work, we introduce a novel floating-point SMT sampler, *DiverFPS*. *DiverFPS* is capable of generating small solution sets that achieve high AST-coverage for SMT formulas in QF_FP and QF_BVFP logics. It effectively alleviates the exploration bottleneck and the redundancy issue present in existing SMT samplers. Extensive experiments on a wide range of standard benchmarks show that, compared to state-of-the-art SMT samplers, *DiverFPS* generates 99.5% AST-coverage solution sets for, on average, 33.6 additional QF_FP benchmarks and 26.6 QF_BVFP benchmarks where existing samplers fail. Moreover, when achieving the same target AST-coverage, the solution sets generated by *DiverFPS* are significantly smaller, with reductions of up to 92.9%. Statistical significance tests were further performed, verifying that the observed improvements are not due to chance but are statistically significant. In addition, our experiments validate the effectiveness of the three novel techniques proposed in this work, *i.e.*, solution-driven restart strategy, context-aware encoding technology, and coverage-driven pruning strategy. Overall, *DiverFPS* advances the state of SMT sampling and provides benefits for applications in software testing and verification.

Data Availability: The implementation of *DiverFPS*, all adopted benchmarks, and detailed experimental results are publicly available at: <https://github.com/ShuangyuLyu/DiverFPS>, and archived at Zenodo: <https://doi.org/10.5281/zenodo.19701170>.

Acknowledgment

This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3307500, in part by the National Natural Science Foundation of China under Grants 62202025 and 62522201, in part by Beijing Natural Science Foundation under Grant L241050, in part by the Young Elite Scientist Sponsorship Program by CAST under Grant YESS20230566, in part by CCF-Huawei Populus Grove Fund under Grant CCF-HuaweiFM2024005, and in part by the Fundamental Research Fund Project of Beihang University.

References

- [1] 1992. *Patriot Missile Defense: Software Problem Led to System Failure at Dhahran, Saudi Arabia*. Technical Report GAO/IMTEC-92-26. http://klabs.org/richcontent/Reports/Failure_Reports/patriot/patriot_gao_145960.pdf
- [2] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2016. The Satisfiability Modulo Theories Library (SMT-LIB). www.SMT-LIB.org. <https://smt-lib.org/>
- [3] Clark Barrett and Cesare Tinelli. 2018. *Satisfiability Modulo Theories*. Springer International Publishing, 305–343. doi:10.1007/978-3-319-10575-8_11
- [4] J. Christopher Beck, Patrick Prosser, and Richard J. Wallace. 2005. Trying Again to Fail-First. In *Recent Advances in Constraints*, Boi V. Faltings, Adrian Petcu, François Fages, and Francesca Rossi (Eds.). Springer Berlin Heidelberg, 41–55. doi:10.1007/11402763_4
- [5] Tim Brys, Madalina M. Drugan, Peter A.N. Bosman, Martine De Cock, and Ann Nowé. 2013. Local search and restart strategies for satisfiability solving in fuzzy logics. In *2013 IEEE International Workshop on Genetic and Evolutionary Fuzzy Systems (GEFS)*. 52–59. doi:10.1109/GEFS.2013.6601055
- [6] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI'08)*. USENIX Association, 209–224. <https://api.semanticscholar.org/CorpusID:2520229>
- [7] Manuel Carrasco, Cristian Cadar, and Alastair F. Donaldson. 2025. Scalable SMT Sampling for Floating-point Formulas via Coverage-guided Fuzzing. In *2025 IEEE/ACM International Conference on Software Testing, Verification and Validation (ICST '25)*. doi:10.1109/ICST62969.2025.10989031
- [8] Supratik Chakraborty, Daniel J. Fremont, Kuldeep S. Meel, Sanjit A. Seshia, and Moshe Y. Vardi. 2015. On Parallel Scalable Uniform SAT Witness Generation. In *Tools and Algorithms for the Construction and Analysis of Systems*, Christel Baier and Cesare Tinelli (Eds.). Springer Berlin Heidelberg, 304–319. doi:10.1007/978-3-662-46681-0_25
- [9] Supratik Chakraborty, Kuldeep S. Meel, Rakesh Mistry, and Moshe Y. Vardi. 2016. Approximate probabilistic inference via word-level counting. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI'16)*. AAAI Press, 3218–3224. doi:10.1609/aaai.v30i1.10416
- [10] Supratik Chakraborty, Kuldeep S. Meel, and Moshe Y. Vardi. 2013. A Scalable Approximate Model Counter. In *Principles and Practice of Constraint Programming*, Christian Schulte (Ed.). Springer Berlin Heidelberg, 200–216. doi:10.1007/978-3-642-40627-0_18
- [11] Supratik Chakraborty, Kuldeep S. Meel, and Moshe Y. Vardi. 2014. Balancing Scalability and Uniformity in SAT Witness Generator. In *Proceedings of the 51st Annual Design Automation Conference*. doi:10.1145/2593069.2593097
- [12] Jianfeng Chen, Xipeng Shen, and Tim Menzies. 2020. Building Very Small Test Suites (with Snap). arXiv:1905.05358 [cs.SE] <https://arxiv.org/abs/1905.05358>
- [13] Qian Chen, Chenqi Cui, Fengjuan Gao, Yu Wang, Ke Wang, and Linzhang Wang. 2025. Solving Floating-Point Constraints with Continuous Optimization. *Proc. ACM Program. Lang.* 9, PLDI, Article 176 (jun 2025), 23 pages. doi:10.1145/3729279
- [14] Edmund Clarke, Daniel Kroening, and Karen Yorav. 2003. Behavioral consistency of C and verilog programs using bounded model checking. In *Proceedings of the 40th annual Design Automation Conference*. 368–371. doi:10.1145/775832.775928
- [15] Emilio Cruciani, Breno Miranda, Roberto Verdecchia, and Antonia Bertolino. 2019. Scalable Approaches for Test Suite Reduction. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 419–429. doi:10.1109/ICSE.2019.00055
- [16] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: an efficient SMT solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'08/ETAPS'08)*. Springer-Verlag, 337–340. doi:10.1007/978-3-540-78800-3_24
- [17] Rafael Dutra, Jonathan Bachrach, and Koushik Sen. 2018. SMTsampler: Efficient Stimulus Generation from Complex SMT Constraints. In *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE Press, 1–8. doi:10.1145/3240765.3240848
- [18] Rafael Dutra, Jonathan Bachrach, and Koushik Sen. 2019. GUIDEDSAMPLER: Coverage-guided Sampling of SMT Solutions. In *2019 Formal Methods in Computer Aided Design (FMCAD)*. 203–211. doi:10.23919/FMCAD.2019.8894251
- [19] Rafael Dutra, Kevin Laeufer, Jonathan Bachrach, and Koushik Sen. 2018. Efficient sampling of SAT solutions for testing. In *Proceedings of the 40th International Conference on Software Engineering (ICSE '18)*. Association for Computing Machinery, 549–559. doi:10.1145/3180155.3180248
- [20] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. 2020. AFL++: combining incremental steps of fuzzing research. In *Proceedings of the 14th USENIX Conference on Offensive Technologies (WOOT'20)*. USENIX Association, Article 10, 1 pages. <https://www.usenix.org/conference/woot20/presentation/fioraldi>
- [21] Zhoulai Fu and Zhendong Su. 2016. XSAT: A Fast Floating-Point Satisfiability Solver. In *Computer Aided Verification*, Swarat Chaudhuri and Azadeh Farzan (Eds.). Springer International Publishing, 187–209. doi:10.1007/978-3-319-41540-

6_11

- [22] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: directed automated random testing. *SIGPLAN Not.* 40, 6 (jun 2005), 213–223. doi:10.1145/1064978.1065036
- [23] Patrice Godefroid, Michael Y. Levin, and David A. Molnar. 2008. Automated Whitebox Fuzz Testing. In *Network and Distributed System Security Symposium*. <https://api.semanticscholar.org/CorpusID:1296783>
- [24] Priyanka Golia, Mate Soos, Sourav Chakraborty, and Kuldeep S. Meel. 2021. Designing Samplers is Easy: The Boon of Testers. In *2021 Formal Methods in Computer Aided Design (FMCAD)*. 222–230. doi:10.34727/2021/isbn.978-3-85448-046-4_31
- [25] Arie Gurfinkel, Temesghen Kahsai, Anvesh Komuravelli, and Jorge A. Navas. 2015. The SeaHorn Verification Framework. In *Computer Aided Verification*. Springer International Publishing, 343–361. doi:10.1007/978-3-319-21690-4_20
- [26] M. Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. 1993. A methodology for controlling the size of a test suite. *ACM Trans. Softw. Eng. Methodol.* 2, 3 (1993), 270–285. doi:10.1145/152388.152391
- [27] Matthias Heizmann, Jochen Hoenicke, and Andreas Podelski. 2013. Software Model Checking for People Who Love Automata. In *Proceedings of the 25th International Conference on Computer Aided Verification - Volume 8044 (CAV 2013)*. Springer-Verlag, 36–52. doi:10.1007/978-3-642-39799-8_2
- [28] Ruben Heradio, David Fernandez-Amoros, José A. Galindo, and David Benavides. 2020. Uniform and scalable SAT-sampling for configurable systems. In *Proceedings of the 24th ACM Conference on Systems and Software Product Line: Volume A - Volume A (SPLC '20)*. Association for Computing Machinery, 11 pages. doi:10.1145/3382025.3414951
- [29] Ruben Heradio, David Fernandez-Amoros, José A. Galindo, David Benavides, and Don Batory. 2022. Uniform and scalable sampling of highly configurable systems. *Empirical Softw. Engg.* 27, 2 (mar 2022), 34 pages. doi:10.1007/s10664-022-10141-6
- [30] Heqing Huang, Peisen Yao, Rongxin Wu, Qingkai Shi, and Charles Zhang. 2020. Pangolin: Incremental Hybrid Fuzzing with Polyhedral Path Abstraction. In *2020 IEEE Symposium on Security and Privacy (SP)*. 1613–1627. doi:10.1109/SP40000.2020.00063
- [31] Yoichiro Iida, Tomohiro Sonobe, and Mary Inaba. 2024. Understand Restart of SAT Solver Using Search Similarity Index (Student Abstract). *Proceedings of the AAAI Conference on Artificial Intelligence* 37, 13 (Jul. 2024), 16236–16237. doi:10.1609/aaai.v37i13.26978
- [32] Seema Jehan and Franz Wotawa. 2023. An Empirical Study of Greedy Test Suite Minimization Techniques Using Mutation Coverage. *IEEE Access* 11 (2023), 65427–65442. doi:10.1109/ACCESS.2023.3289073
- [33] Baharin A. Jodat, Abhishek Chandar, Shiva Nejati, and Mehrdad Sabetzadeh. 2024. Test Generation Strategies for Building Failure Models and Explaining Spurious Failures. *ACM Trans. Softw. Eng. Methodol.* 33, 4, Article 93 (2024), 32 pages. doi:10.1145/3638246
- [34] M. Ammar Ben Khadra, Dominik Stoffel, and Wolfgang Kunz. 2017. goSAT: floating-point satisfiability as global optimization. In *Proceedings of the 17th Conference on Formal Methods in Computer-Aided Design (FMCAD '17)*. FMCAD Inc, 11–14. doi:10.23919/FMCAD.2017.8102235
- [35] Nathan Kitchen. 2010. *Markov Chain Monte Carlo Stimulus Generation for Constrained Random Simulation*. University of California, Berkeley. <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2010/EECS-2010-165.html>
- [36] Nathan Kitchen and Andreas Kuehlmann. 2007. Stimulus generation for constrained random simulation. In *2007 IEEE/ACM International Conference on Computer-Aided Design*. 258–265. doi:10.1109/ICCAD.2007.4397275
- [37] Daniel Kroening and Michael Tautschnig. 2014. CBMC – C Bounded Model Checker. In *Tools and Algorithms for the Construction and Analysis of Systems, Erika Ábrahám and Klaus Havelund (Eds.)*. Springer Berlin Heidelberg, Berlin, Heidelberg, 389–391. doi:10.1007/978-3-642-54862-8_26
- [38] libfuzzer. [n. d.]. *LibFuzzer*. <http://lvm.org/docs/LibFuzzer.html>
- [39] Daniel Liew, Cristian Cadar, Alastair F. Donaldson, and J. Ryan Stinnett. 2019. Just fuzz it: solving floating-point constraints using coverage-guided fuzzing. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2019)*. Association for Computing Machinery, 521–532. doi:10.1145/3338906.3338921
- [40] Chu-Ti Lin, Kai-Wei Tang, Jiun-Shiang Wang, and Gregory M. Kapfhammer. 2017. Empirically evaluating Greedy-based test suite reduction methods at different levels of test suite complexity. *Sci. Comput. Program.* 150, C (2017), 1–25. doi:10.1016/j.scico.2017.05.004
- [41] Dongge Liu, Gidon Ernst, Toby Murray, and Benjamin I. P. Rubinstein. 2021. Legion: best-first concolic testing. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE '20)*. Association for Computing Machinery, 54–65. doi:10.1145/3324884.3416629
- [42] Chuan Luo, Shaowei Cai, Wei Wu, Zhong Jie, and Kaile Su. 2015. CCLS: An Efficient Local Search Algorithm for Weighted Maximum Satisfiability. *IEEE Transactions on Computers* 64, 7 (2015), 1830–1843. doi:10.1109/TC.2014.2346196
- [43] Chuan Luo, Jianping Song, Qiyuan Zhao, Binqi Sun, Junjie Chen, Hongyu Zhang, Jinkun Lin, and Chunming Hu. 2025. Solving the t-Wise Coverage Maximum Problem via Effective and Efficient Local Search-Based Sampling. *ACM Trans.*

- Softw. Eng. Methodol.* 34, 1, Article 13 (dec 2025), 64 pages. doi:10.1145/3688836
- [44] Chuan Luo, Binqi Sun, Bo Qiao, Junjie Chen, Hongyu Zhang, Jinkun Lin, Qingwei Lin, and Dongmei Zhang. 2021. LS-Sampling: An Effective Local Search based Sampling Approach for Achieving High t-wise Coverage. In *Proceedings of ESEC/FSE 2021*. 1081–1092. doi:10.1145/3468264.3468622
 - [45] Chuan Luo, Qiyuan Zhao, Shaowei Cai, Hongyu Zhang, and Chunming Hu. 2022. SamplingCA: Effective and Efficient Sampling-Based Pairwise Testing for Highly Configurable Software Systems. In *Proceedings of ESEC/FSE 2022*. 1185–1197. doi:10.1145/3540250.3549155
 - [46] Valentin J. M. Manès, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J. Schwartz, and Maverick Woo. 2018. The Art, Science, and Engineering of Fuzzing: A Survey. *IEEE Transactions on Software Engineering* 47 (2018), 2312–2331. doi:10.1109/TSE.2019.2946563
 - [47] Matthew W. Moskewicz, Conor F. Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik. 2001. Chaff: engineering an efficient SAT solver. In *Proceedings of the 38th Annual Design Automation Conference (DAC '01)*. Association for Computing Machinery, 530–535. doi:10.1145/378239.379017
 - [48] Alexander Nadel. 2011. Generating diverse solutions in SAT. In *Proceedings of the 14th International Conference on Theory and Application of Satisfiability Testing (SAT'11)*. Springer-Verlag, 287–301. doi:10.1007/978-3-642-21581-0_23
 - [49] Aina Niemetz and Mathias Preiner. 2023. Bitwuzla. In *Computer Aided Verification - 35th International Conference, CAV 2023, Paris, France, July 17-22, 2023, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 13965)*, Constantin Enea and Akash Lal (Eds.). Springer, 3–17. doi:10.1007/978-3-031-37703-7_1
 - [50] Dangdang Niu, Bin Liu, Minghao Yin, and Yupeng Zhou. 2023. A new local search algorithm with greedy crossover restart for the dominating tree problem. *Expert Systems with Applications* 229 (2023), 120353. doi:10.1016/j.eswa.2023.120353
 - [51] Raphael Noemmer and Roman Haas. 2020. An Evaluation of Test Suite Minimization Techniques. In *Software Quality: Quality Intelligence in Software and Systems Engineering*, Dietmar Winkler, Stefan Biffl, Daniel Mendez, and Johannes Bergmann (Eds.). Springer International Publishing, 51–66. doi:10.1007/978-3-030-35510-4_4
 - [52] Mingyu Park, Hoon Jang, Taejoon Byun, and Yunja Choi. 2020. Property-based Testing for LG Home Appliances using Accelerated Software-in-the-Loop Simulation. In *2020 IEEE/ACM 42nd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 120–129. doi:10.1145/3377813.3381346
 - [53] Matan I. Peled, Bat-Chen Rothenberg, and Shachar Itzhaky. 2023. SMT Sampling via Model-Guided Approximation. In *Formal Methods*, Marsha Chechik, Joost-Pieter Katoen, and Martin Leucker (Eds.). Springer International Publishing, 74–91. doi:10.1007/978-3-031-27481-7_6
 - [54] Sebastian Poeplau and Aurélien Francillon. 2020. Symbolic execution with SYMCC: don't interpret, compile!. In *Proceedings of the 29th USENIX Conference on Security Symposium (SEC'20)*. USENIX Association, 18 pages. <https://www.usenix.org/conference/usenixsecurity20/presentation/poeplau>
 - [55] Jaideep Ramachandran, Corina Păsăreanu, and Thomas Wahl. 2015. Symbolic Execution for Checking the Accuracy of Floating-Point Programs. *SIGSOFT Softw. Eng. Notes* 40, 1 (feb 2015), 1–5. doi:10.1145/2693208.2693242
 - [56] Clément Robert, Jérémie Guiochet, Hélène Waeselynck, and Luca Vittorio Sartori. 2021. TAF: a Tool for Diverse and Constrained Test Case Generation. In *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*. 311–321. doi:10.1109/QRS54544.2021.00042
 - [57] Federica Sarro, Mark Harman, Yue Jia, and Yuanyuan Zhang. 2018. Customer Rating Reactions Can Be Predicted Purely using App Features. In *Proceedings of RE 2018*. 76–87. doi:10.1109/RE.2018.00018
 - [58] Nico Schiller, Xinyi Xu, Lukas Bernhard, Nils Bars, Moritz Schloegel, and Thorsten Holz. 2025. Novelty Not Found: Exploring Input Shadowing in Fuzzing through Adaptive Fuzzer Restarts. *ACM Trans. Softw. Eng. Methodol.* 34, 3, Article 85 (feb 2025), 32 pages. doi:10.1145/3712186
 - [59] Arijit Shaw and Kuldeep S. Meel. 2024. CSB: A Counting and Sampling Tool for Bit-vectors. In *Proceedings of the 22nd International Workshop on Satisfiability Modulo Theories co-located with the 36th International Conference on Computer Aided Verification (CAV 2024), Montreal, Canada, July, 22-23, 2024 (CEUR Workshop Proceedings, Vol. 3725)*, Giles Reger and Yoni Zohar (Eds.). CEUR-WS.org, 36–43. <https://api.semanticscholar.org/CorpusID:271333193>
 - [60] Sohil Lal Shrestha, Shafiqul Azam Chowdhury, and Christoph Csallner. 2022. SLNET: a redistributable corpus of 3rd-party simulink models. In *Proceedings of the 19th International Conference on Mining Software Repositories (Pittsburgh, Pennsylvania) (MSR '22)*. Association for Computing Machinery, New York, NY, USA, 237–241. doi:10.1145/3524842.3528001
 - [61] András Vargha and Harold D. Delaney. 2000. A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics* 25, 2 (2000), 101–132. doi:10.3102/10769986025002101
 - [62] Yiyuan Wang, Dantong Ouyang, Minghao Yin, Liming Zhang, and Yonggang Zhang. 2018. A restart local search algorithm for solving maximum set k-covering problem. *Neural Comput. Appl.* 29, 10 (may 2018), 755–765. doi:10.1007/s00521-016-2599-7

[63] Michal Zalewski. [n. d.]. *Technical whitepaper for afl-fuzz*. http://lcamtuf.coredump.cx/afl/technical_details.txt

Received 2025-09-12; accepted 2025-12-22